Introduction to NLP, Exercise 4

Sabine Weber

Publication: Nov 10, 2026
Submission: Nov 17, 2026
Discussion: Nov 19, 2026

- **Deadline:** The submission needs to be done via VC until the end of the day mentioned on the PDF. A late submission is not possible.
- **Groups:** You are allowed to work in groups. A group can consist of maximally three members. The members can change in each submission. The names that you put on the PDF are what we consider as the group.
- **Grading:** We plan to offer 11 exercise sheets. For each sheet, you can achieve 20 points. From the 220 points altogether, we will consider 176 points to be a perfect participation (if, for some reason, we offer fewer assignments, this number of required points will be reduced proportionally). The assignment submissions will contribute up to 20 % of the grade as bonus points.
- **Submission:** Only submit one PDF file. Program code should be part of that PDF file. Put all names and email addresses of your group members on the first page of this PDF file. Please do submit the PDF only once (not multiple times by multiple group members).
- **Sources:** We recommend to not use any "AI" or large language model for creating the solution to any subtask/task as a whole. You can use other sources (documentation, papers, stackexchange, ...). If you decide to use an "AI" or LLM (like ChatGPT, Hawki) for a part of your solution, provide the information which system you used, provide the complete prompt and the complete answer by the automatic system (in German or English) in an appendix of your submission. I need to be able to understand what your own contribution on top of the automatic result was. You only receive points for your own contribution on top of what you got from the source you provide. If you use an "AI" and do not provide the required reference or do not make that use transparent, we will consider this plagiarism.
- **Plagiarism:** When two groups submit solutions that are copied from each other or from a third source that is not made transparent in the submission, all these groups do not receive any points. The same holds if one group copies from a source that is no provided in the submission. The whole submission will be considered invalid, not only the task in which we detected the plagiarism. No points will be given for the whole submission. These sanctions do not exclude further measures.

Do you have questions? Send me an email (sabine.weber@uni-bamberg.de) or use the forum.

Task 1 - Pen & Paper (10 points)

A: Create dependency trees for the following sentences. Use Universal Dependencies (UD) relation labels. For each word, identify its head and dependency relation. Discuss and explain your result.

1. It is a nice night.
2. This crap game is over a garage in Fifty-second Street.
3. Nobody ever takes the newspapers she sells.
4. He is a tall, skinny guy with a long, sad, mean-looking kisser, and a mournful voice.

B: Draw dependency trees for the following sentences. Identify any dependencies whose arcs cross if the words are written in their original linear order. Why does this pose a problem for dependency parsing?

1. Which book did you say that Mary bought?
2. The person I spoke to yesterday has already left.
3. What did John give Mary?

C: Consider the following ambiguous sentences:

1. I saw her duck.
2. She discussed the problem with the professor.
3. The chicken is ready to eat.

Create two dependency trees for each sentence, corresponding to the two possible interpretations. Discuss and explain your result.

Task 2 - Programming (10 points)

In this exercise, you will implement a simple transition-based dependency parser using the arc-standard parsing algorithm. The parser will construct a dependency tree by applying a sequence of transitions to a sentence.

1. Represent the input

Represent each sentence as a list of tokens. Each token should contain a word and its POS tag. You may use a POS tagger or assign the POS tags manually. For example:

```
[
    ("The", "DT"),
    ("dog", "NN"),
    ("chased", "VBD"),
    ("the", "DT"),
    ("cat", "NN")
]
```

The parser should maintain three data structures:

- A **stack**, initially containing a special ROOT symbol.
- A **buffer**, initially containing all sentence tokens.
- A list of **dependency arcs**, initially empty.

Each dependency arc should contain a head and a dependent. For this exercise you do **not** need to include a dependency relation label.

2. Implement the transitions
Implement the following three transitions:

- **SHIFT**: Remove the first token from the buffer and push it onto the stack.
- **LEFT-ARC**: Add an arc from the second item on the stack to the top item on the stack. Then remove the top item from the stack.
- **RIGHT-ARC**: Add an arc from the top item on the stack to the second item on the stack. Then remove the second item from the stack.

For example, if the stack contains `[ROOT, chased, dog]` and the transition is `LEFT-ARC`, the parser adds the dependency `chased --> dog` and removes `dog` from the stack.

3. Implement the parser
Write a function that takes a sentence as input and returns a dependency tree. The function should:

1. Initialize the stack, buffer, and dependency arc list.
2. Repeatedly apply transitions until the buffer is empty and only ROOT remains on the stack.
3. Return the resulting dependency arcs.

To decide which transition to apply, use a simple rule-based strategy. Use the POS tags of the tokens to define rules. E.g., nouns are likely subjects, verbs are likely predicates, determiners modify nouns, and adjectives modify nouns.
Your parser should be able to handle simple sentences with a subject, a verb, and an object, as well as determiners and adjectives.

4. Test your parser
Create five sentences of your own and run them through your parser. Choose sentences that contain different syntactic structures:

1. A sentence with a subject, verb, and object.
2. A sentence with an adjective and a determiner.
3. A sentence with a prepositional phrase.
4. A sentence with a coordination.

5. A sentence containing a subordinate or relative clause.

You may use a POS tagger or assign the POS tags manually. For each sentence:

1. Print the sequence of transitions applied by your parser.
2. Print the resulting dependency arcs.
3. Draw or visualize the resulting dependency tree.
4. Check whether the resulting tree is linguistically plausible.

5. Reflection
Answer the following questions:

1. Did your parser produce any incorrect dependency relations? Give examples and explain what likely went wrong.
2. What kinds of sentences or constructions were difficult to parse using your rules?
3. What would you need to add or change to make the parser work on sentences that you have not seen before?